

Lab Practice

Practice Problems Related to PL/SQL

CSE 4410

RELATIONAL DATABASE MANAGEMENT SYSTEM LAB

June 5, 2026

Schema

```
1 CREATE TABLE agents (  
2     agent_id          SERIAL PRIMARY KEY,  
3     agent_name        VARCHAR(50),  
4     role              VARCHAR(20),    -- Duelist, Initiator,  
5         Controller, Sentinel  
6     rank_rating       INTEGER,  
7     contracts_completed INTEGER,  
8     bonus_points      NUMERIC(10,2)  
9 );  
10 CREATE TABLE matches (  
11     match_id          SERIAL PRIMARY KEY,  
12     map_name          VARCHAR(30),  
13     available_slots   INTEGER,  
14     match_status      VARCHAR(20)    -- OPEN, FULL  
15 );  
16 CREATE TABLE player_match (  
17     registration_id    SERIAL PRIMARY KEY,  
18     agent_id           INTEGER REFERENCES agents(agent_id),  
19     match_id           INTEGER REFERENCES matches(match_id),  
20     registration_date   DATE DEFAULT CURRENT_DATE  
21 );  
22 CREATE TABLE agent_performance (  
23     performance_id     SERIAL PRIMARY KEY,  
24     agent_id           INTEGER REFERENCES agents(agent_id),  
25     kills              INTEGER,  
26     deaths            INTEGER,  
27     assists           INTEGER,  
28     combat_score       INTEGER  
29 );  
30 CREATE TABLE bonus_audit (  
31     audit_id          SERIAL PRIMARY KEY,  
32     agent_id          INTEGER,  
33     old_bonus         NUMERIC(10,2),  
34     new_bonus         NUMERIC(10,2),  
35     action_type       VARCHAR(50),  
36     action_date       TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
37 );  
38 CREATE TABLE promotion_review (  
39     review_id          SERIAL PRIMARY KEY,  
40     agent_id           INTEGER,  
41     avg_combat_score   NUMERIC(8,2),  
42     tier               VARCHAR(50),  
43     review_date        DATE DEFAULT CURRENT_DATE  
44 );
```

Tasks

Task 1 — Valorant Combat Efficiency Analyzer

Write a PL/pgSQL **procedure**:

```
analyze_agent_performance(p_agent_id INT)
```

that reads all performance records of the given agent from the `agent_performance` table and performs the following steps.

Step 1 — Compute per-match efficiency. For every match record, calculate:

$$\text{Efficiency} = (\text{Kills} + \text{Assists}) - \text{Deaths}$$

Step 2 — Aggregate statistics. Across all matches determine the Highest Efficiency, the Lowest Efficiency, and the Average Efficiency.

Step 3 — Assign a Performance Rating using the table below:

Average Efficiency	Rating
≥ 20	ELITE
15 – 19.99	PRO
10 – 14.99	ADVANCED
< 10	ROOKIE

Step 4 — Print the result. The expected output format is:

```
Agent: Jett
Highest Efficiency : 28
Lowest Efficiency  : 4
Average Efficiency : 15.33
Performance Rating : ELITE
```

Task 2 — Valorant Match Registration System

When agents register for a match, slot counts must be updated automatically.

Create a **trigger** on the `player_match` table that fires **BEFORE INSERT**. The trigger function must enforce the following rules in order:

Rule 1 — Full match. If `available_slots = 0` for the target match, raise:

Match is already full.

Rule 2 — Duplicate registration. If the same agent is already registered for the same match in `player_match`, raise:

Agent already registered for this match.

Rule 3 — Decrement slot. After the above checks pass, decrease `available_slots` by 1 in the `matches` table.

Rule 4 — Update match status. Set `match_status` to 'FULL' when `available_slots` reaches 0; otherwise set it to 'OPEN'.

Example:

```
-- Before insert:
```

```
Match 12 | Available Slots = 1 | Status = OPEN
```

```
-- After successful insert:
```

```
Match 12 | Available Slots = 0 | Status = FULL
```

Task 3 — Valorant Agent Promotion Review

Using an **explicit cursor** with `OPEN`, `FETCH`, and `CLOSE`, process every agent in the `agents` table one by one.

Step 1 — Compute average combat score. For each agent, calculate the average `combat_score` from all rows in `agent_performance`.

Step 2 — Assign Promotion Tier:

Average Combat Score	Tier
≥ 300	Radiant Candidate
250 – 299	Immortal Candidate
200 – 249	Ascendant Candidate
< 200	Needs Training

Step 3 — Insert into `promotion_review`.

Step 4 — Print per-agent output using `GET DIAGNOSTICS`:

```
Processing: Jett
```

```
Average Score: 315.00
```

```
Tier: Radiant Candidate
```

```
Inserted: 1 row
```